lex and yacc — Developer Guide

This repo carries its own scanner generator (`lex/lex.c`) and its own LALR(1) parser generator (`yacc/yacc.c`). Both are written in the C subset that our own compiler `cc` accepts, and both are compiled by `cc` to .NET IL. They generate C, which `cc` compiles. Nothing outside the repo is involved: no flex, no bison, no native C toolchain.

Every language front end here is built with them: Pascal, Modula-2/Oberon-2, Tiny C++, QBasic, Forth, Fortran 90, COBOL, Ada, Smalltalk, Lua, AWK, Coil, `bc` , and the shell `ilsh` . This document is the reference for the two tools, plus the things you only find out by reading their source.

Two worked examples live under `examples/lexyacc/` : `calc/` (a four-function calculator) and `minishell/` (a shell with `;` and `&&`).

Contents

1. The pipeline
2. The .l file format
3. The .y file format
4. Performance and conflicts
5. Reusing the toolchain for your own language
6. The worked examples
7. Limits, at a glance

---

1. The pipeline

Both generators read their specification from **stdin** and write C to **stdout**. There are no input or output filename arguments. The full build of a language called `mylang` is four commands:

```
CC="dotnet src/Cc/bin/Release/net10.0/cc.dll"

dotnet yacc/yacc.dll < mylang/mylang.y > out/parse.c    # grammar  → LALR(1) parser
dotnet lex/lex.dll   < mylang/mylang.l > out/scan.c     # patterns → scanner
cat out/parse.c out/scan.c > out/mylang_src.c           # ORDER MATTERS
$CC out/mylang_src.c -o out/mylang.exe --exe            # C subset → .NET IL
```

`build_all.sh` runs exactly this pattern once per language, and `shell/build.sh` does the same for `ilsh` with three extra `.c` files appended to the `cat` .

Why parser-then-scanner

The two generated files are not independent translation units; they are concatenated into one C file, and `cc` requires things to be declared before they are used. The parser is the file that declares what the

scanner needs:

- **Token codes.** `yacc` emits one `enum` per named terminal: `enum { NUMBER = 257 };`. Named tokens are numbered from 257 upwards in declaration order; a character literal such as `'+'` uses its character code. The scanner's actions say `return NUMBER;`, so those enums must already exist.
- **`yylval`.** `yacc` emits `int yylval; int yyval;`. The scanner's actions assign to `yylval`; they never declare it.
- **`yytranslate`** maps a token code back to an internal symbol id, so the parser can accept whatever `yylex()` returns.

Reverse the `cat` and every token name in the scanner is an undefined identifier. There is no header file and no `-d` flag: the enum in the generated parser *is* the interface.

In the other direction the parser calls `yylex()`, which the scanner defines later in the same file. Declare it in the `.y` prologue (`int yylex();`) so the forward reference resolves.

What ends up in the single C file

```
+-- from mylang.y ------------------------------------------+
| your %{ ... %} prologue, verbatim                         |
| enum { TOK = 257 }; ... one per named terminal            |
| yytranslate[], yyact[], yyarg[], yygoto[], yyplen[],      |
|   yyplhs[]  (the LALR tables)                             |
| int yylval; int yyval; int yyss[8192]; int yyvs[8192];    |
| int yyerror(char *s)         (default; yours can win)     |
| int yyparse(void)            (the LR engine + actions)    |
| everything after the second %%, verbatim (incl. main)     |
+-----------------------------------------------------------+
+-- from mylang.l ------------------------------------------+
| your %{ ... %} prologue, verbatim                         |
| yyop[], yyx[], yyy[], yycls[]  (the regex VM program)     |
| char *yytext; int yyleng;                                 |
| char *yy_buf; int yy_pos; int yy_len; int yy_started;     |
| void yy_scan_string(char *s); void yy_init(void);         |
| int yylex(void)              (the VM + your actions)      |
| everything after the second %%, verbatim                  |
+-----------------------------------------------------------+
```

`cc` produces `out/mylang.dll` plus a launcher `out/mylang.exe`. Add `--icon icons/foo.png` for an icon, or `--dll` instead of `--exe` for a library callable from C#/VB.NET.

---

2. The .l file format

```
%{
  C code copied verbatim into the generated scanner (helpers, globals, state)
%}

NAME    pattern            /* named definitions, referenced later as {NAME} */

%%
pattern    { action }
pattern    { action }
%%

C code copied verbatim after yylex()
```

Both `%%` lines must sit alone on their line. The second one and the user-code section may be omitted.

Definitions section

- `%{ ... %}` copies lines verbatim to the top of the generated scanner. The delimiters are recognised line-wise, so braces inside are safe.
- `NAME pattern` defines a macro. A reference `{NAME}` inside a later pattern is textually replaced by `(pattern)`, parentheses included, so `DIGIT [0-9]` plus `{DIGIT}+` behaves as `([0-9])+`.
- Any other line starting with `%` is **silently skipped**. That means `%option`, `%s`, `%x`, `%array` are accepted and ignored, not diagnosed. There are no start conditions (see below), so an ignored `%x` will not do what you hoped.

Rules section

One rule per line: a pattern, then whitespace, then an action.

The pattern ends at the first space or tab that is **not** inside `[ ... ]`, not inside `" ... "`, and not escaped. To match a literal space use `" "`, `[ ]`, or `\ `.

An action is either `{ ... }` (which may span lines) or the rest of the line. An action that does not `return` falls through and scanning continues, which is how whitespace and comment rules work:

```
[ \t]+        { }
"#"[^\n]*     { }
```

Regex syntax that is implemented

Verified against `lex/lex.c` (`parse_atom`, `parse_class`, `parse_rep`, `parse_alt`):

Construct	Meaning
`abc`	literal characters
`.`	any character except newline (`\n`)
`[abc]` `[a-z0-9_]`	character class, with ranges
`[^ \t\n]`	negated class (complement over all 256 codes)
`*` `+` `?`	zero or more, one or more, optional
`|`	alternation
`( ... )`	grouping
`" ... "`	literal string; escapes inside are honoured
`\n \t \r \f \v \0`	escapes; `\X` for any other X is literal X
`{NAME}`	expand a named definition

Escapes work inside classes too, including as range endpoints (`[\0-\37]`). Postfix operators may be stacked (`a*?` parses, though it means the same as `a*`).

Regex syntax that is NOT implemented

Do not use these; several fail silently rather than erroring.

- **Start conditions.** No `%s` / `%x` , no `<COND>pattern` , no `BEGIN( ... )` . A leading `<` in a pattern is parsed as a literal `<` . Context-sensitive lexing has to be done with your own state variables in the prologue. `shell/shell.l` is the worked example: it keeps `sc_cmd` / `sc_fname` / `sc_needin` flags so reserved words are only reserved in command position.
- **Anchors.** `^` outside a class and `$` are literal characters, not beginning/end-of-line anchors.
- **Counted repetition.** `a{2,5}` is not repetition. `{` starts a definition reference; if the name does not resolve, the braces stay literal.
- **Trailing context.** `r/s` is not lookahead; `/` is a literal slash.
- `<<EOF>>` **rules.**
- `unput()` , `yyless()` , `yymore()` , `input()` , `REJECT` , `yywrap()` . There is no pushback mechanism at all. If you need to un-read, restructure the pattern or keep a one-token queue in your own code.

Matching semantics

- **Longest match wins.** All rules run in parallel as one Thompson NFA (a regex VM with `CHAR` / `ANY` / `CLASS` / `SPLIT` / `JMP` / `MATCH` instructions); the longest overall match is taken.
- **Ties go to the earliest rule** in the file. So put `"&&"` before any rule that could match a single `&` , and keep a catch-all `.` last.
- **A rule that can match the empty string never fires.** Matches of length 0 are rejected, so `a*` as a whole pattern is dead code.
- **An unmatched character is skipped silently.** If no rule matches at the current position, `yy_pos` advances by one and scanning continues with no diagnostic. Always end the rules with a catch-all

so bad input reaches the parser as a token it can complain about:

```
. { return yytext[0]; }
```

Scanner interface

Symbol	Type	Notes
`yylex()`	`int`	returns a token code; `0` at end of input, which is `$end`
`yytext`	`char *`	the matched text, NUL-terminated, valid until the next `yylex()`
`yyleng`	`int`	its length
`yylval`	`int`	set this to pass a semantic value to the parser (declared by `yacc`)
`yy_scan_string(s)`		scan the NUL-terminated buffer `s` instead of stdin
`yy_buf`, `yy_pos`, `yy_len`	`char *`, `int`, `int`	the buffer, the cursor, the length; readable and writable
`yy_started`	`int`	0 until a buffer is chosen; `yy_scan_string` sets it
`yy_init()`		called automatically on the first `yylex()` if no buffer was set: slurps **all of stdin** into a `malloc`'d buffer

Two consequences worth planning for. First, a compiler driver normally calls `rt_slurp()` then `yy_scan_string()`; only a filter should rely on the implicit stdin path. Second, a REPL calls `yy_scan_string()` once per line and `yyparse()` once per line, so a syntax error costs one line and not the session (`bc/bc.y` and `examples/lexyacc/calc/calc.y` both do this).

`yy_pos` and `yy_len` are ordinary globals, so a scanner action can peek ahead with `yy_buf[yy_pos]` or consume raw text by advancing `yy_pos` itself. That is the only substitute for `input()` / `unput()`.

Two gotchas in `lex` itself

Braces inside string literals in an action break the action reader. `lex` counts `{` and `}` with no regard for quoting (unlike `yacc`, whose action reader does skip string and character literals). This rule:

```
"a"   { printf((int)"open brace: {\n"); return 1; }
```

consumes the rest of the file into rule 0's action and emits broken C. Keep braces balanced inside actions (`"{ ... }"` is fine), build such strings in a helper function in the `%{ ... %}` prologue, or assemble them from `%c` and a character code.

No bounds are checked. `yacc` reports its own overflows with a message and exit code 2; `lex` does not check anything. Exceeding a `lex` limit (see section 6) corrupts memory quietly.

---

3. The .y file format

```
%{
  C code copied verbatim into the generated parser (helpers, globals, prototypes)
%}
%token NAME1 NAME2
%left  '+' '-'
%start rulename

%%
nonterm : symbol symbol   { action }
        | symbol          { action }
        ;
%%

C code copied verbatim after yyparse() (usually main())
```

Declarations

Declaration	Effect
`%token A B 'c'`	declare terminals, no precedence
`%left A '+'`	declare terminals, left-associative, next precedence level
`%right A '='`	as above, right-associative
`%nonassoc A`	as above; using the operator twice in a row is an error
`%start rule`	the start symbol; defaults to the LHS of the first rule
anything else `% ...`	**silently ignored** (`%type` , `%union` , `%expect` , ...)

Each `%left` / `%right` / `%nonassoc` line creates one precedence level, and **later lines bind tighter**, as in classic yacc. `%token` creates no level.

There is no `%union` and no `%type` : the semantic value type is always `int` . Anything larger travels as a heap address cast to `int` . This is the single biggest stylistic consequence for the grammars in this repo, and it is why you see `(char *)$1` , `(int)"literal"` and box/unbox helpers everywhere.

> **Declarations do not continue onto the next line.** `%token` , `%left` , `%right` and `%nonassoc` read only to the end of their own line. This:
>
> `%token C D E`
>
> declares `C` and `D` . `E` is silently dropped, and later becomes an *undeclared nonterminal* with no productions the moment a rule mentions it. Repeat the keyword instead:
>
> `%token C D %token E`

Terminals may be named (`NUMBER`) or character literals (`'+'`, `'\n'`). A character literal used only inside a rule is auto-declared as a terminal; a name used only inside a rule is auto-declared as a nonterminal.

Rules

`lhs : rhs ;` with `|` between alternatives. An empty alternative is legal and conventionally written `/* empty */`. Actions are `{ ... }` and are placed at the end of the alternative.

Inside an action:

Form	Expands to	Meaning
`$$`	`yyval`	the value of the left-hand side
`$N`	`yyvs[yybase+N-1]`	the value of the Nth right-hand-side symbol
`$0`	`yyvs[yybase-1]`	the *inherited* value of the symbol immediately left of this rule

The **default action** is `$$ = $1` when the rule has at least one symbol, and `$$ = 0` for an empty rule. So a pass-through alternative needs no action.

`%prec TOKEN` anywhere in an alternative sets that production's precedence explicitly. Without it, a production's precedence is that of its **rightmost terminal**.

`yyparse()` returns 0 on accept and 1 on error. It uses a fresh stack each call, so calling it repeatedly (once per line, once per file) is safe and is how the REPLs here work.

Three gotchas, in order of how much they will cost you

No mid-rule actions. Writing an action in the middle of an alternative does *not* create the anonymous marker nonterminal real yacc would create. `yacc.c` reads every `{ ... }` it meets into the same slot, so all but the **last** action are silently discarded, and the last one runs at the end of the rule. This:

```
s : A { printf("mid1\n"); } B { printf("mid2\n"); } C ;
```

emits one action, `printf("mid2\n")`, fired after `C` is seen. `$1` / `$2` / `$3` still number the grammar symbols `A` / `B` / `C`, because no marker was inserted. There is no warning. Use an explicit marker rule instead (below).

No negative stack access. `$-1`, `$-2` and so on are not recognised: the `$` is copied through literally and `cc` rejects the result. `$0` *is* recognised, and is the whole basis of the marker trick.

`$` substitution happens inside string and character literals too. The rewriter walks the action text without tracking quotes, so

```
{ printf((int)"marker sees $0 = %d\n", $0); }
```

prints `marker sees yyvs[yybase+-1] = 3`. Never put `$` followed by a digit or another `$` inside a literal in an action. (`lex` does not rewrite `$` at all, so this applies only to `.y` files.)

The marker trick

Every non-trivial front end in this repo needs to run code *part way* through a rule, usually to emit a loop header before the body is parsed. With mid-rule actions unavailable, the idiom is an **empty nonterminal placed where the action should fire**, whose action reads `$0`.

`$0` is `yyvs[yybase-1]`. For an empty rule `len == 0`, so `yybase == yysp` and `$0` is the value sitting immediately below the marker on the value stack: the inherited value of whatever symbol precedes the marker in the enclosing rule.

A minimal, real example. `mark` sits between two numbers and reads the first:

```
%token NUM
%%
prog : /* empty */ | prog line ;
line : NUM mark NUM '\n'  { printf((int)"rule: $1=%d $3=%d\n", $1, $3); } ;
mark : /* empty */        { printf((int)"marker sees %d\n", $0); } ;
```

On the input `3 4` this prints, in this order:

```
marker sees 3
rule: $1=3 $3=4
```

The marker reduced as soon as the parser had `NUM` on the stack and `4` in the lookahead, which is exactly the "part way through" hook. The marker itself has a value (`$$`), and a rule can read it later as an ordinary `$N`, so markers double as scratch slots.

The marker does not have to be empty. `lua/lua.y` uses the loop keyword itself:

```
stmt : WHILE expr wdo block END   { apc("}\n"); }
     ;
wdo  : DO { apc(F1("while (truthy(%s)) {\n", ec($0))); } ;
```

`wdo` has one symbol, so `yybase == yysp - 1` and `$0` is the value below `DO`, which is `expr`. The `while ( ... ) {` header is emitted before `block` is parsed; the outer rule closes the brace afterwards. `cpp/cpp.y` uses the same pattern to thread a declaration's base type into each declarator (`setdt : { g_basetype = $0; }`), `awk/awk.y` for `if` / `while` / `for` headers, `ada/ada.y` for call names and loop variables.

Two practical notes:

- A marker adds a state and a reduction, so the parser must be able to decide to reduce it with only one token of lookahead. If placing a marker introduces conflicts, move it or split the rule.
- Because `$0` reaches *below* the current rule, a marker is only safe if every context that can reach it has the same symbol to its left. Give each site its own marker nonterminal rather than reusing one; that is why `lua/lua.y` has `wdo`, `ifthen`, `eithen`, `feq`, `fcomma`, `fhi`, `fink`, `fiter` and not one generic `mark`.

`yyerror`, `yylex`

`yacc` always emits a default `int yyerror(char *s) { printf("%s\n", s); return 0; }`. If you define your own `yyerror` in the `%{ ... %}` prologue, **yours wins**: `cc` keeps the first definition it sees, and the prologue is emitted before the generated one. Verified; no warning either way.

There is no `error` token and no error recovery. `yyparse()` reports and returns 1. Grammars here get useful messages by tracking the line number in the scanner and printing it from a custom `yyerror`.

Declare `int yylex();` in the prologue. Add prototypes there for any function your actions call but define later in the user-code section.

---

4. Performance and conflicts

What the generator does

`yacc.c` builds the canonical LR(1) item-set collection but merges states as it goes: a state is identified by its **LR(0) core** (the item set ignoring lookaheads), and when a `goto` reaches a state whose core already exists, the new lookaheads are unioned into it and the state is re-queued so the extra lookaheads propagate. That is LALR(1) **by construction**, and it keeps the state count at LR(0) size instead of exploding into canonical LR(1) states.

Practical consequence: the tables are LALR(1), so the classic LALR caveat applies. Merging cores can introduce reduce/reduce conflicts that canonical LR(1) would not have. In practice this bites only on grammars that distinguish two contexts purely by lookahead, and the fix is the same as for real yacc: factor the grammar so the decision is visible earlier.

Reading the diagnostics

`-v` prints to stderr (the parser itself goes to stdout, so redirect stdout as usual):

```
$ dotnet yacc/yacc.dll -v < examples/lexyacc/calc/calc.y > out/calc_parse.c
yacc: nsym=16 nprod=14
yacc: LALR states=23 reproc=40 gotos=143 itemscanned=1307
yacc: conflicts=0
```

- `nsym` / `nprod` against the hard limits of 512 and 600.
- `states` against the limit of 3000.
- `reproc` is how many times a state was (re)processed; `gotos` the number of closures computed; `itemscanned` the total item-visits. `itemscanned` is the real cost driver. Real grammars in this repo:

grammar	nsym	nprod	states	itemscanned	conflicts
`examples/lexyacc/minishell/minishell.y`	10	11	12	98	0
`examples/lexyacc/calc/calc.y`	16	14	23	1307	0

grammar	nsym	nprod	states	itemscanned	conflicts
`shell/shell.y`	35	33	62	5376	1
`lua/lua.y`	91	104	191	179791	3
`pascal/pascal.y`	153	191	386	142750	1

All of these generate in well under a second. `itemscanned` grows roughly with ambiguity, not with grammar size: `lua/lua.y` is smaller than `pascal/pascal.y` but scans more items because its expression grammar is ambiguous and resolved by precedence.

- `conflicts` is a **count only**. There is no `y.output`, no per-state listing, and no indication of which rules collided. To locate one, comment rules out and watch the count, or bisect the grammar into a smaller file that still reports it.

Shift/reduce vs reduce/reduce

A **shift/reduce** conflict is a state where the parser could either push the lookahead or reduce a completed production. The archetype is `expr : expr '+' expr` with `1 + 2 * 3` on the input: with `expr '+' expr` on the stack and `'*'` in the lookahead, both are legal.

A **reduce/reduce** conflict is a state where two different completed productions both apply. That almost always means the grammar is genuinely ambiguous or two nonterminals overlap, and precedence cannot help. Resolution here is "the earlier production in the file wins", which is a coin toss dressed as a rule: treat any reduce/reduce conflict as a grammar bug.

How precedence resolves them

For a shift/reduce conflict, `yacc.c` compares the precedence of the **lookahead terminal** with the precedence of the **production** (its `%prec`, else its rightmost terminal):

Situation	Resolution
production precedence > terminal	reduce
production precedence < terminal	shift
equal, terminal is `%left`	reduce
equal, terminal is `%right`	shift
equal, terminal is `%nonassoc`	error (the action is removed)
either side has no precedence	**counted as a conflict**, shift wins

That last row is the one to watch: an unresolved shift/reduce conflict is not an error, it silently defaults to shift. For the dangling-else and similar cases that is the behaviour you want, and one or two conflicts in a real grammar are normal (`shell/shell.y` and `pascal/pascal.y` each carry exactly one). A count in the dozens means the grammar is not saying what you think.

For a reduce/reduce conflict, precedence is not consulted at all: the lower-numbered production wins, and the conflict is counted.

Ambiguous grammars, and the fix

An ambiguous expression grammar is fine *if* every operator carries a precedence. `examples/lexyacc/calc/calc.y` is deliberately ambiguous:

```
expr : expr '+' expr | expr '-' expr | expr '*' expr | expr '/' expr |  ...  ;
```

With its four precedence lines it reports `conflicts=0`. Delete only those four lines and the same grammar reports `conflicts=24`. Nothing else changed; the declarations were carrying the whole disambiguation.

The alternative is a **stratified** grammar, one nonterminal per precedence tier, which needs no declarations at all:

```
expr   : expr '+' term | expr '-' term | term ;
term   : term '*' factor | term '/' factor | factor ;
factor : '-' factor | '(' expr ')' | NUMBER | NAME ;
```

Both are conflict-free. Trade-offs: the stratified form is self-documenting, has no reliance on precedence resolution, and produces more states and one reduction per tier per operand; the precedence-tiered form is far shorter and easier to extend but is only correct as long as every operator is declared.

When conflicts are in the dozens, the fix is almost always one of:

1. Add or correct precedence declarations, remembering that later lines bind tighter and `%token` gives no precedence at all.
2. Stratify the expression grammar into tiers.
3. Left-factor rules with a common prefix so the decision point moves later, where one token of lookahead is enough.
4. Move the ambiguity into the scanner. `shell/shell.l` decides whether `if` is a keyword or a plain word using its own state, so the grammar never has to.

Left recursion is preferred throughout (`list : list item | item`): an LR parser handles it with a constant-depth stack, and the alternative costs stack depth for no benefit. The parser stack is 8192 entries.

---

5. Reusing the toolchain for your own language

Minimal skeleton

`mylang.l`:

```
%{
/* scanner helpers and state. Token names and yylval come from the parser's
 * output, which is concatenated first; g_line is declared in mylang.y. */
%}
DIGIT   [0-9]
ALPHA   [A-Za-z_]
%%
[ \t\r]+                        { }
"//"[^\n]*                      { }
\n                              { g_line++; }
{DIGIT}+                        { yylval = atoi((int)yytext); return NUMBER; }
{ALPHA}({ALPHA}|{DIGIT})*       { yylval = (int)strdup((int)yytext); return NAME; }
":="                            { return ASSIGN; }
[-+*/()<>;,]                    { return yytext[0]; }
.                               { return yytext[0]; }
%%
```

`mylang.y`:

```
%{
/* prologue: globals, helpers, prototypes for anything the actions call */
int g_line;
void yyerror(char *m) { printf((int)"mylang:%d: %s\n", g_line, (int)m); }
int yylex();
%}
%token NUMBER NAME ASSIGN
%left '+' '-'
%left '*' '/'
%start program
%%
program : /* empty */
        | program stmt
        ;
stmt    : NAME ASSIGN expr ';'   { /* emit an assignment */ }
        ;
expr    : NUMBER                 { $$ = $1; }
        | NAME                   { $$ = lookup((char *)$1); }
        | expr '+' expr          { $$ = binop('+', $1, $3); }
        | expr '*' expr          { $$ = binop('*', $1, $3); }
        | '(' expr ')'           { $$ = $2; }
        ;
%%
/* user code: main(), plus the functions the actions called */
```

The two prologues land in one translation unit, so a global shared between scanner and parser is declared **once, in the** `.y` (which is concatenated first) and simply used in the `.l`: that is `g_line` above. `cc` does tolerate the same global being declared in both prologues, but it keeps the first declaration and **discards the second one's initialiser**, so `int g_line = 1;` in the `.l` would silently leave `g_line` at 0.

That skeleton builds and runs as written. Fed `x := 1 + 2 * 3;` with `binop` tracing its operator, it reports `conflicts=0` and prints:

```
binop *
binop +
assign x = 6
```

The interpreter case

If the actions *do* the work, you are finished. `bc/bc.y` evaluates in its actions and its `main()` is a dozen lines: read a line, `yy_scan_string`, `yyparse`, repeat. No temporary files, no `cc`, nothing to shell out to.

`examples/lexyacc/minishell/minishell.y` is the same shape with one extra step: the actions build a small tree and `main()` walks it.

The compiler-driver case

A compiler front end here emits C text, writes it to a file, and shells out to `cc`. This is the pattern every language directory uses; `lua/lua.y` is the clearest one to read. Runtime calls come from `CRuntime`:

Call	Purpose
`rt_slurp(path)`	read a whole file; returns a `char *` as int, or 0 on failure
`rt_repo()`	absolute path of the repo root (the directory holding `build_all.sh`)
`fopen/fputs/fclose`	write the generated C
`sh_run(argv, argc)`	`Process.Start` + `WaitForExit`; returns the exit code, 127 if not found

`argv` is the address of an array of `int`, each element a `char *` to a NUL-terminated argument, element 0 the program.

```
void setext(char *p, char *e)   /* replace the extension in place */
{
    int n = strlen(p), i = n - 1;
    while (i > 0 && p[i] ≠ '.' && p[i] ≠ '\\' && p[i] ≠ '/') i--;
    if (p[i] == '.') p[i + 1] = 0; else strcat(p, ".");
    strcat(p, e);
}

int main(int argc, char **argv)
{
    if (argc < 2) { printf((int)"usage: mylang <file.ml> [-o out] [--dll]\n"); return 1; }
    char *in = (char *)argv[1]; char *o = 0; int dll = 0; int i;
    for (i = 2; i < argc; i++)
    {
        if (strcmp((char *)argv[i], "-o") == 0 && i + 1 < argc) o = (char *)argv[++i];
        else if (strcmp((char *)argv[i], "--dll") == 0) dll = 1;
    }

    char outp[1024], cpath[1024];
    if (o) strcpy(outp, o); else { strcpy(outp, in); setext(outp, "exe"); }
    strcpy(cpath, outp); setext(cpath, "c");        /* the intermediate C file */

    char *src = (char *)rt_slurp((int)in);
    if (!src) { printf((int)"mylang: cannot read %s\n", (int)in); return 1; }

    yy_scan_string((int)src);                       /* parse from memory, not stdin */
    if (yyparse() ≠ 0) return 1;                    /* actions filled the code buffers */

    int f = fopen((int)cpath, (int)"w");            /* write the generated C */
    emit_prelude(f);
    fputs((int)g_code, f);
    fclose(f);

    char cc[1100]; char *repo = (char *)rt_repo();  /* shell out to our C compiler */
    sprintf((int)cc, (int)"%s\\src\\Cc\\bin\\Release\\net10.0\\cc.exe", (int)repo);
    int av[8]; int n = 0;
    av[n++] = (int)cc;
    av[n++] = (int)cpath;
    av[n++] = (int)"-o";  av[n++] = (int)outp;
    av[n++] = dll ? (int)"--dll" : (int)"--exe";
    int rc = sh_run((int)av, n);
    if (rc == 0) printf((int)"mylang: %s → %s\n", (int)in, (int)outp);
    else         printf((int)"mylang: cc failed (%d)\n", rc);
    return rc;
}
```

Then add four lines to `build_all.sh` in the shape every other language uses.

C subset idioms to absorb first

Read `bc/bc.y` and one larger grammar before writing your own. The recurring requirements of the C that `cc` accepts:

- Cast pointers to `int` when passing them to variadic or runtime functions: `printf((int)"%s\n", (int)s)`, `atof((int)yytext)`, `strdup((int)yytext)`.
- Declare loop variables before the loop: `int i; for (i = 0; ...)`, never `for (int i = 0; ...)`.
- Semantic values are `int`. Box anything else on the heap and cast the address. `bc/bc.y` boxes doubles; `lua/lua.y` boxes a tagged `Val`; `shell/shell.y` boxes AST nodes.
- Arrays inside structs are fine (`struct Words { int n; int w[64]; };`).
- Build strings with small helpers rather than inline concatenation; the `j2` / `F1` / `F2` / `F3` helpers in `lua/lua.y` are the house style.
- Global fixed-size arrays are used in preference to dynamic growth almost everywhere. Size them generously; nothing bounds-checks them.

---

6. The worked examples

Two complete programs under `examples/lexyacc/`, each about 100 lines of specification, with their own README. The sessions below are real.

calc

A four-function calculator with variables. An ambiguous expression grammar plus four precedence lines.

```
CC="dotnet src/Cc/bin/Release/net10.0/cc.dll"
dotnet yacc/yacc.dll -v < examples/lexyacc/calc/calc.y > out/calc_parse.c
dotnet lex/lex.dll      < examples/lexyacc/calc/calc.l > out/calc_scan.c
cat out/calc_parse.c out/calc_scan.c > out/calc_src.c
$CC out/calc_src.c -o out/calc.exe --exe
```

Input, then output:

```
1 + 2 * 3        →  7
(1 + 2) * 3      →  9
2 - 3 - 4        →  -5
x = 10           →  10
y = x * 4 + 2    →  42
y / 7            →  6
-x + 1           →  -9
z                →  0
1 / 3            →  0.3333333333
```

`2 - 3 - 4 == -5` is `%left`; `1 + 2 * 3 == 7` is the later precedence line; `-x + 1 == -9` is `%prec UMINUS`. Removing the four precedence lines and changing nothing else takes the same grammar from `conflicts=0` to `conflicts=24`.

minishell

A shell: `WORD+` commands, `;` sequencing, `&&` short-circuit, external execution through `sh_run`.

```
dotnet yacc/yacc.dll -v < examples/lexyacc/minishell/minishell.y > out/ms_parse.c
dotnet lex/lex.dll      < examples/lexyacc/minishell/minishell.l > out/ms_scan.c
cat out/ms_parse.c out/ms_scan.c > out/minishell_src.c
$CC out/minishell_src.c -o out/minishell.exe --exe
```

Input:

```
cmd /c echo hello from minishell
cmd /c echo first ; cmd /c echo second
cmd /c exit 0 && cmd /c echo left succeeded
cmd /c exit 1 && cmd /c echo never printed
# a comment line
nosuchprog
exit
```

Output:

```
hello from minishell
first
second
left succeeded
nosuchprog: command not found
```

Ten productions, `conflicts=0`, no precedence declarations: the `list` / `andlist` / `cmd` / `words` layering makes `;` and `&&` unambiguous structurally.

---

7. Limits, at a glance

`yacc` checks the entries marked (checked) and exits with status 2 and a message on stderr. Everything else, in both tools, overflows silently.

yacc

Limit	Value
symbols (terminals + nonterminals)	512 (checked)

Limit	Value
productions	600 (checked)
symbols on one right-hand side	24
terminal symbol ids	must be < 256: lookaheads are packed into 8 bits of an item, so declare all tokens up front, which `%token` does naturally
named token codes	from 257 upwards, in declaration order
action text, per production	4090 characters (checked)
prologue / user-code section	200000 characters each (checked)
LALR states	3000 (checked)
items per state	5000 (checked)
core items per state	1024 (checked)
input `.y` file	400000 characters
parser stack depth at run time	8192

lex

Limit	Value
rules	256
named definitions	128 (name 63 chars, pattern 511 chars)
pattern text, per rule	512 characters, 1024 after `{NAME}` expansion
action text, per rule	2048 characters
prologue	20000 characters
user-code section	40000 characters
regex VM instructions, all rules combined	40000
character classes	512
input `.l` file	400000 characters
`yytext` at run time	8191 characters

Behaviour to keep in mind

unknown `%` declaration	silently ignored, both tools
`%token` / `%left` /... continuation line	silently ignored

mid-rule action	all but the last silently discarded; the last runs at end of rule
`$-N`	not supported; `$0` is
`$N` inside a string literal in a `.y` action	rewritten
`{` / `}` inside a string literal in a `.l` action	miscounted; swallows the rest of the file
unmatched input character	skipped silently by the scanner
empty-matching lex rule	never fires
unresolved shift/reduce conflict	counted, shift wins
unresolved reduce/reduce conflict	counted, earlier production wins
user `yyerror` in the prologue	overrides the generated default